

Laporan Tugas Besar 2 IF3270 Pembelajaran Mesin

Convolutional Neural Network dan Recurrent Neural Network



Anggota Kelompok:

Hasri Fayadh (NIM: 13523156)

I Made Wiweka Putera (NIM: 13523160)

Fudhail Fayyadh (NIM: 18223121)

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

1. Deskripsi Persoalan

Tugas Besar II IF3270 Pembelajaran Mesin bertujuan untuk memberikan pemahaman mendalam mengenai implementasi Convolutional Neural Network (CNN) dan Recurrent Neural Network (RNN). Fokus utama dari tugas ini adalah membangun modul forward propagation untuk CNN, Simple RNN, dan LSTM sepenuhnya dari awal (*from scratch*) tanpa bergantung pada abstraksi tingkat tinggi dari framework machine learning, dengan hanya menggunakan *library* komputasi numerik. Tugas ini dibagi menjadi dua domain persoalan: Image Classification dan Image Captioning.

Pada persoalan pertama (Image Classification), model CNN dibangun untuk mengklasifikasikan dataset Intel Image Classification yang terdiri dari enam kategori *landscape*. Implementasi *from scratch* ini mencakup pembuatan fungsionalitas matematis untuk layer Conv2D (menggunakan shared parameter), LocallyConnected2D (menggunakan non-shared parameter), Pooling, Flatten, dan Dense. Objektif pada tahap ini adalah mengeksplorasi dan membandingkan dampak penggunaan parameter sharing terhadap makro F1-score, jumlah total parameter jaringan, serta efisiensi kinerja secara keseluruhan.

Persoalan kedua mengeksplorasi pipeline Image Captioning melalui implementasi arsitektur encoder-decoder pada dataset Flickr8k. Sistem ini dirancang berdasarkan arsitektur "Show and Tell" (Vinyals et al., 2015), di mana sebuah CNN pretrained digunakan secara statis sebagai encoder untuk mengekstraksi vektor fitur high-dimensional dari sebuah gambar. Vektor fitur tersebut kemudian diintegrasikan ke dalam arsitektur sekuensial (RNN atau LSTM) menggunakan metode pre-inject. Dalam metode ini, vektor gambar diproyeksikan melalui layer Dense dan dimasukkan ke dalam jaringan rekuren pada timestep $t=-1$, tepat sebelum sekuens token teks disalurkan. Analisis pada tahap ini dititikberatkan pada perbandingan matematis dan komputasional antara model Simple RNN dan LSTM, secara spesifik mengevaluasi bagaimana masing-masing arsitektur menangani fenomena vanishing gradient dan retensi memori jangka panjang saat melakukan generasi teks.

2. Pembahasan

2.1. Penjelasan Implementasi

1. Arsitektur Modular & OOP

Seluruh layer diimplementasikan sebagai class independen secara modular. Setiap class memiliki atribut untuk menyimpan parameter (weights, bias) dan metode forward() yang menggunakan NumPy untuk komputasi tensor.

2. Implementasi CNN (Convolutional Neural Network)

A. Layer Konvolusi & Spasial

- Conv2DLayer: Mengimplementasikan konvolusi 2D dengan parameter sharing. Menggunakan `np.einsum('nhwc,hwck->nk')` untuk efisiensi perkalian patch dengan kernel.
 - Atribut: kernel $[kH, kW, C_{in}, C_{out}]$, bias $[C_{out}]$.
 - Padding: Mendukung same (dengan `np.pad`) dan valid.
- LocallyConnected2DLayer: Berbeda dengan Conv2D, layer ini tidak menggunakan parameter sharing. Setiap posisi spasial memiliki bobot unik.
 - Atribut: kernel $[outH \times outW, kH \times kW \times C_{in}, C_{out}]$, bias $[outH \times outW, C_{out}]$.
- Pooling Layers:
 - MaxPooling2DLayer & AveragePooling2DLayer: Menggunakan sliding window dengan `np.max` atau `np.mean` pada dimensi spasial.
 - GlobalAveragePooling2DLayer & GlobalMaxPooling2DLayer: Mereduksi seluruh dimensi spasial (H, W) menjadi nilai tunggal per channel menggunakan `axis=(1, 2)`.
- FlattenLayer: Mengubah tensor 4D (N, H, W, C) menjadi 2D (N, $H \times W \times C$) menggunakan row-major (C-order) agar konsisten dengan output Keras.

B. Layer Dense & Aktivasi

- DenseLayer: Melakukan transformasi linear $Y = XW + b$.
 - Atribut: weights $[dim_{in}, dim_{out}]$, bias $[dim_{out}]$.
- Fungsi Aktivasi:
 - ReLU: `np.maximum(0, x)`.
 - Softmax: Menggunakan pengurangan nilai maksimum ($x - np.max(x)$) sebelum eksponensial untuk stabilitas numerik (numerical stability).

3. Implementasi RNN & LSTM (Image Captioning)

Implementasi decoder menggunakan metode pre-inject, di mana fitur gambar dimasukkan sebagai token pertama (x_{-1}) untuk menginisialisasi konteks sebelum memulai sekuens teks.

A. Komponen Layer

- **EmbeddingLayer:** Berfungsi sebagai matrix lookup table untuk memetakan indeks kata ke vektor kontinu.
 - Atribut: `weights [vocab_size, embed_dim]`.
 - Forward: Mengambil baris matriks berdasarkan indeks `self.weights[x]`.
- **DenseProjectionLayer (Image Projection):** Mentransformasi fitur CNN (2048-dim dari InceptionV3) ke dalam dimensi embedding agar kompatibel dengan input RNN/LSTM.
- **DenseOutputLayer:** Mentransformasi hidden state terakhir menjadi distribusi probabilitas seluruh kosakata melalui aktivasi Softmax.

B. Recurrent Cells

- **SimpleRNNCell:** Mengupdate hidden state menggunakan rumus $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$.
- **LSTMCell:** Mengelola long-term memory melalui empat gates yang digabungkan dalam satu operasi dot product untuk efisiensi:
 - Gates: Forget gate (f_t), Input gate (i_t), Cell candidate (g_t), dan Output gate (o_t).
 - Komputasi:
 1. $Z = (x_t \cdot W_x) + (h_{t-1} \cdot W_h) + b$.
 2. Split Z menjadi 4 bagian untuk masing-masing gate.
 3. $c_t = f_t \odot c_{t-1} + i_t \odot g_t$ (Update cell state).
 4. $h_t = o_t \odot \tanh(c_t)$ (Update hidden state).
 - Atribut: $W_x [\text{dim}_{in}, 4 \times \text{dim}_{hidden}]$, $W_h [\text{dim}_{hidden}, 4 \times \text{dim}_{hidden}]$, $b [4 \times \text{dim}_{hidden}]$.

4. Struktur Data & Dimensi Tensor

Semua input dan output dipertahankan dalam format `np.float32`. Operasi batching didukung dengan dimensi pertama sebagai `batch_size`.

- Image Features: `[batch, 2048]`.
- Word Indices: `[batch, seq_len]`.
- RNN/LSTM Hidden State: `[batch, hidden_dim]`.
- Output Distribution: `[batch, vocab_size]`.

5. Fungsi Load Weights

Fungsi `load_weights()` pada setiap class dirancang untuk memetakan bobot dari format Keras (misalnya, urutan kernel dan bias) ke dalam atribut NumPy yang sesuai. Hal ini memastikan bahwa meskipun pelatihan dilakukan di Keras, logika eksekusi forward pass 100% independen dan dapat diverifikasi keakuratannya melalui perbandingan output numerik ($\text{Epsilon} < 1e-6$).

2.2. Penjelasan Forward Propagation

2.2.1. CNN

Forward propagation pada CNN adalah suatu proses yang mengalirkan data input (berupa tensor gambar) secara sekuensial melewati setiap layer hingga dapat menghasilkan prediksi akhir yang dimana setiap layer akan melakukan transformasi matematis. Output dari layer sebelumnya akan menjadi input untuk layer berikutnya.

a. Conv2DLayer (Shared Parameter)

Konvolusi 2d adalah inti dari sebuah arsitektur CNN. ide dasarnya adalah pola visual yang relevan seperti tepi (edge), tekstur, atau bentuk dapat muncul dimana saja dalam sebuah gambar. Maka dari itu, tidak efisien jika semua posisi spasial memiliki detector polanya tersendiri. Conv2d mengatasi hal ini dengan menggunakan konsep parameter sharing yang dimana satu set filter yang sama diaplikasikan secara berulang kepada seluruh posisi spasial pada input.

Dimensi Tensornya adalah sebagai berikut:

- Input x : batch dari gambar masing masing berukuran $H \times W$ dengan C_{in} sebagai channel
- Kernel W : bobot yang dipelajari berukuran $K_h \times K_w$ per channel input untuk menghasilkan C_{out}
- Bias b : satu nilai bias per channel outputnya
- output: $H_{out} = \left\lfloor \frac{H_{padded} - K_{Ht}}{S_H} \right\rfloor + 1$, $W_{out} = \left\lfloor \frac{W_{paded} - K_W}{S_W} \right\rfloor + 1$

Operasi matematisnya adalah sebagai berikut:

Untuk setiap sampel n , posisi output (i,j) , dan filter k :

$$out[n, i, j, k] = b[k] + \sum_{c=0}^{C_{in}-1} \sum_{p=0}^{K_H-1} \sum_{q=0}^{K_W-1} x[n, i \cdot S_W + p, q, c] \cdot W[p, q, c, k]$$

Operasi ini pada dasarnya adalah dot product antara patch lokal dari input dan kernel, dilakukan untuk setiap posisi (i,j)

(i,j) dan setiap filter k . Hasilnya dijumlahkan dengan bias $b[k]$.

Implementasi NumPy:

```
for i in range(out_H):
    for j in range(out_W):
        h_start = i * self.stride_h
        w_start = j * self.stride_w
        patch = x[:, h_start:h_start+kH, w_start:w_start+kW, :]
        # patch: (N, kH, kW, C_in), kernel: (kH, kW, C_in, C_out)
        out[:, i, j, :] = np.einsum('nhwc,hwck->nk', patch, self.kernel) + self.bias

    out = self.activation(out)
    return out[0] if squeeze else out
```

Maksud dari notasi Einstein 'nhwc,hwck->nk' adalah untuk setiap sampel n dan filter output k , jumlahkan atas semua dimensi h , w , dan c . ini secara efisien menggantikan 3 nested loop dalam satu operasi vektor

b. LocallyConnected2DLayer (Non-Shared Parameter)

LocallyConnected2D merupakan generalisasi dari Conv2D yang melepaskan asumsi parameter sharing. Pada layer ini, setiap posisi spasial output (i,j) memiliki kernel yang sepenuhnya independen dan unik ini berarti layer tidak memaksakan fitur yang sama namun sebaliknya layer membebaskan mempelajari detector yang berbeda untuk setiap region gambar. Hal ini memiliki konsekuensi yang dimana jumlah parameternya akan jauh lebih besar dibandingkan dengan conv2D karena setiap posisi spasialnya memiliki parameternya tersendiri.

Dimensi Tensor:

- Kernel W: satu kernel per posisi output
- Bias b: satu bias per posisi output

Operasi matematisnya adalah sebagai berikut:

Untuk setiap posisi output ke-p, filter k dan sampel n:

$$out[n, i, j, k] = \sum_{c=0}^{K_H \cdot K_W \cdot C_{in} - 1} patch_flat[n, f] \cdot W[p, f, k] + b[p, k]$$

di mana patch_flat adalah patch lokal yang telah di-*flatten* menjadi vektor 1D berukuran $K_H \cdot K_W \cdot C_{in}$

Implementasi NumPy

```
for i in range(out_H):
    for j in range(out_W):
        p = i * out_W + j
        h_start = i * self.stride_h
        w_start = j * self.stride_w
        patch = x[:, h_start:h_start+kH, w_start:w_start+kW, :]
        # patch: (N, kH, kW, C_in) → flatten: (N, kH*kW*C_in)
        patch_flat = patch.reshape(N, -1)
        # kernel[p]: (kH*kW*C_in, C_out)
        out[:, i, j, :] = patch_flat @ self.kernel[p] + self.bias[p]

    out = self.activation(out)
    return out[0] if squeeze else out
```

self.kernel[p] mengambil kernel unik untuk posisi p, berukuran $K_H \cdot K_W \cdot C_{in} \cdot C_{out}$. Perkalian matriks patch_flat @ self.kernel[p] menghasilkan output berukuran N, C_{out} untuk posisi tersebut.

c. Pooling Layers

Setelah layer konvolusi menghasilkan feature map, dimensi spasialnya masih sangat besar

dan rentan terhadap variasi posisi kecil (spatial sensitivity). Pooling layer hadir untuk dua tujuan utama:

- Reduksi dimensi spasial: Mengurangi ukuran $H \times W$ sehingga jumlah parameter dan komputasi pada layer berikutnya berkurang secara drastis.
- Translational invariance: Membuat representasi lebih robust terhadap pergeseran kecil posisi fitur dalam gambar jika sebuah tepi bergeser sedikit, nilai pooling tetap relatif sama.

MaxPooling2D:

MaxPooling mengambil nilai maksimum dari setiap window. Operasi ini secara selektif meneruskan sinyal terkuat dari setiap region, mempertahankan fitur yang paling menonjol sambil mengabaikan yang lemah.

Implementasi NumPy adalah sebagai berikut:

```
for i in range(out_H):
    for j in range(out_W):
        h_start = i * self.stride_h
        w_start = j * self.stride_w
        patch = x[:, h_start:h_start+self.pool_h, w_start:w_start+self.pool_w, :]
        out[:, i, j, :] = np.max(patch, axis=(1, 2))

return out[0] if squeeze else out
```

Reduksi `axis=(1, 2)` mengambil nilai maksimum sepanjang dimensi H dan W dari patch, menghasilkan tensor (N,C) per posisi output.

AveragePoolig2D:

AveragePooling mengambil nilai rata-rata dari setiap window. Berbeda dengan MaxPooling yang hanya meneruskan satu sinyal terkuat, AveragePooling mempertahankan kontribusi seluruh piksel dalam window secara merata, menghasilkan representasi yang lebih "halus" (smoothed).

Implementasi Numpynya adalah sebagai berikut:


```

for i in range(out_H):
    for j in range(out_W):
        h_start = i * self.stride_h
        w_start = j * self.stride_w
        patch = x[:, h_start:h_start+self.pool_h, w_start:w_start+self.pool_w, :]
        out[:, i, j, :] = np.mean(patch, axis=(1, 2))

return out[0] if squeeze else out

```

GlobalAveragePooling2D:

GAP adalah kasus ekstrem dari AveragePooling di mana window mencakup seluruh dimensi spasial sekaligus. Hasilnya adalah satu nilai per channel, merepresentasikan rata-rata aktivasi global dari seluruh feature map tersebut.

Implementasi NumPynya adalah sebagai berikut:

```

def forward(self, x):
    squeeze = x.ndim == 3
    if squeeze:
        x = x[np.newaxis]
    out = np.mean(x, axis=(1, 2))
    return out[0] if squeeze else out

```

d. FlattenLayer

Mengubah tensor 4D (N,H,W,C) menjadi 2D (N,H×W×C) agar dapat diproses oleh layer Dense.

```

def forward(self, x):
    squeeze = x.ndim == 3
    if squeeze:
        x = x[np.newaxis]
    out = x.reshape(x.shape[0], -1)
    return out[0] if squeeze else out

```

Penggunaan C-order (row-major / default NumPy) memastikan urutan elemen konsisten dengan perilaku Flatten pada Keras.

e. Fungsi Aktivasi

Aktivasi diterapkan element-wise setelah operasi linear pada Conv2D maupun Dense.

Fungsi	Formula
ReLU	$f(x) = \max(0, x)$
Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$
Tanh	$f(x) = \tanh(x)$
Softmax	$f(x_i) = \frac{e^{x_i - \max(x)}}{1 + \sum e^{x_j - \max(x)}}$
Linear	$f(x) = x$

2.2.2. RNN

Proses Ekstraksi Bobot Layer Dense (Projection dan Output)

Implementasi decoder berbasis RNN menggunakan dua layer Dense (fully connected) untuk pemetaan dimensi yang berbeda. Pertama, DenseProjectionLayer berfungsi sebagai proyektor fitur visual. Layer ini menerima vektor fitur ekstraksi CNN (seperti InceptionV3) yang memiliki dimensi (Batch, 2048). Melalui operasi transformasi linear standar, fitur tersebut diproyeksikan ke dalam ruang dimensi embedding menggunakan persamaan $Y = X \cdot W + b$, di mana W adalah matriks bobot berdimensi (2048, embed_dim) dan b adalah bias berdimensi (embed_dim). Hasil keluaran matriks berdimensi (Batch, embed_dim) ini kemudian diubah bentuknya (reshaping) menjadi (Batch, 1, embed_dim) untuk merepresentasikan timestep $t=-1$ pada sekuens input.

Kedua, DenseOutputLayer memproses hidden state akhir dari RNN pada setiap timestep dan memetakannya ke dalam ruang dimensi kosakata. Hasil transformasi linear di layer ini diproses lebih lanjut menggunakan fungsi aktivasi Softmax untuk menghasilkan distribusi

probabilitas dari seluruh vocabulary dengan dimensi akhir (Batch, vocab_size).

Proses Lookup Matriks pada Embedding Layer

Pemrosesan token teks dilakukan oleh EmbeddingLayer. Berbeda dengan layer komputasional lainnya, layer ini tidak menggunakan operasi perkalian matriks (matrix multiplication). Sebaliknya, proses embedding direalisasikan melalui direct index-based row lookup. Layer menerima input berupa array integer dua dimensi dengan dimensi (Batch, Sequence_Length), di mana setiap integer merupakan indeks representasi kata. Operasi lookup dilakukan secara langsung terhadap matriks bobot embedding yang berdimensi (vocab_size, embed_dim). Hasil dari pemetaan indeks ini adalah tensor tiga dimensi yang merepresentasikan fitur linguistik kontinu dengan bentuk (Batch, Sequence_Length, embed_dim).

Proses Input Sequence (Pre-Inject Method)

Arsitektur RNN pada sistem image captioning ini mengadopsi pendekatan Pre-inject. Dalam arsitektur ini, representasi gambar tidak digabungkan secara paralel di setiap timestep, melainkan diperlakukan sebagai token awal (pre-injeksi) yang mendahului sekuens teks. Tensor gambar yang telah diproyeksikan dari dimensi (Batch, 1, embed_dim) digabungkan pada indeks ke-0 dari sekuens teks berdimensi (Batch, Sequence_Length, embed_dim). Penggabungan ini dilakukan menggunakan operasi np.concatenate pada sumbu sekuens, sehingga menghasilkan satu tensor sekuens terpadu yang memuat konteks visual pada awal sekuens dengan dimensi akhir (Batch, Sequence_Length + 1, embed_dim).

Loop Propagasi Hidden State pada Sel SimpleRNN

Evaluasi temporal pada SimpleRNNCell dieksekusi melalui iterasi sekuensial sepanjang sequence axis dari timestep $t=0$ hingga $t=N$. Pada timestep awal, hidden state h_0 diinisialisasi secara eksplisit sebagai matriks nol dengan dimensi (Batch, hidden_size). Pembaruan hidden state pada setiap timestep t dikomputasi menggunakan formulasi berikut:

$$h_t = \tanh(x_t \cdot W_x + h_{t-1} \cdot W_h + b)$$

Di mana x_t adalah vektor input pada waktu ke- t dengan dimensi (Batch, embed_dim), W_x adalah matriks bobot input berdimensi (embed_dim, hidden_size), h_{t-1} adalah hidden state dari timestep sebelumnya, W_h adalah matriks bobot rekuren berdimensi (hidden_size, hidden_size), dan b

merepresentasikan vektor bias berdimensi (hidden_size). Selama proses iterasi, nilai hidden state h_t dari setiap timestep dikumpulkan dan disimpan dalam sebuah tensor array. Kumpulan hidden state ini kemudian diteruskan secara sekuensial menuju DenseOutputLayer untuk proses decode probabilitas kosakata.

2.2.3. LSTM

Operasi Transformasi Gabungan (Z-Matrix Calculation)

Dalam implementasi from-scratch menggunakan NumPy, efisiensi komputasi pada forward propagation LSTM dicapai dengan melakukan transformasi linear untuk keempat gate secara simultan dalam satu operasi dot product. Pendekatan ini meminimalkan overhead pemanggilan fungsi matriks yang berulang. Komputasi matriks gabungan Z didefinisikan melalui persamaan blok berikut:

$$Z = (x_t \cdot W_x) + (h_{t-1} \cdot W_h) + b$$

Dalam persamaan tersebut, tensor x_t memiliki dimensi (Batch, embed_dim) yang merepresentasikan input pada timestep t. Matriks bobot input W_x memiliki dimensi (embed_dim, 4 x hidden_size), sedangkan matriks bobot rekuren W_h memiliki dimensi (hidden_size, 4 x hidden_size). Vektor hidden state sebelumnya, h_{t-1} , memiliki dimensi (Batch, hidden_size). Hasil dari operasi ini adalah matriks Z dengan dimensi (Batch, 4 x hidden_size), yang menampung informasi pra-aktivasi untuk forget gate, input gate, cell candidate, dan output gate.

Pemisahan Tensor (Splitting) dan Aktivasi Gate

Matriks Z yang dihasilkan kemudian dipisahkan secara horizontal menjadi empat sub-matriks independen dengan dimensi masing-masing (Batch, hidden_size). Pemisahan ini dilakukan menggunakan teknik array slicing atau fungsi np.split untuk memperoleh Z_f , Z_i , Z_g dan Z_o . Setiap sub-matriks tersebut kemudian dikenakan fungsi aktivasi secara elemen-per-elemen (element-wise) untuk menentukan aktivasi setiap gate:

- Forget gate: $f_t = \sigma(Z_f)$
- Input gate: $i_t = \sigma(Z_i)$
- Cell candidate: $g_t = \tanh(Z_g)$
- Output gate: $o_t = \sigma(Z_o)$

Di sini, σ merepresentasikan fungsi aktivasi Sigmoid yang memetakan nilai ke dalam rentang $[0, 1]$, yang berfungsi sebagai mekanisme kontrol aliran informasi pada sel.

Pembaruan Cell State (c_t) dan Hidden State (h_t)

Setelah seluruh aktivasi gate diperoleh, dilakukan pembaruan terhadap status internal sel. Berbeda dengan transformasi linear sebelumnya, operasi ini menggunakan produk Hadamard (perkalian elemen-per-elemen, disimbolkan dengan $\odot$) untuk memanipulasi informasi pada tingkat unit hidden. Persamaan untuk pembaruan cell state (c_t) dan hidden state (h_t) adalah sebagai berikut:

$$c_t = (f_t \odot c_{t-1}) + (i_t \odot g_t)$$

$$h_t = o_t \odot \tanh(c_t)$$

Pembaruan c_t melibatkan interaksi antara forget gate dengan cell state sebelumnya (c_{t-1}) serta interaksi antara input gate dengan cell candidate (g_t). Hidden state h_t kemudian dihasilkan dengan memfilter hasil aktivasi non-linear dari cell state yang baru menggunakan output gate.

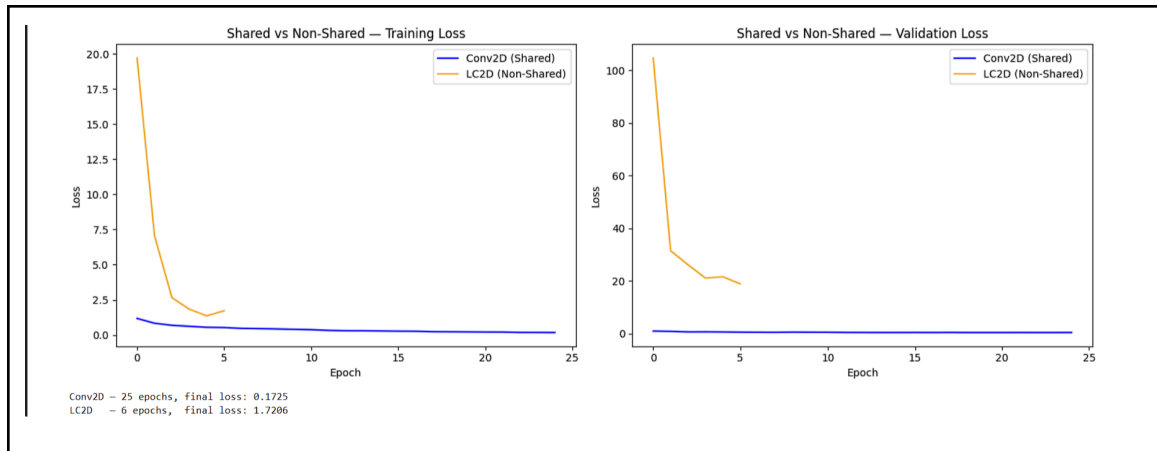
Inisialisasi Temporal

Pada awal pemrosesan sekuens data ($t=0$), sistem tidak memiliki informasi konteks dari timestep sebelumnya. Oleh karena itu, hidden state awal (h_0) dan cell state awal (c_0) secara eksplisit diinisialisasi sebagai matriks nol. Kedua tensor inisialisasi tersebut memiliki dimensi (Batch, hidden_size), yang berfungsi sebagai nilai dasar sebelum diterimanya input pertama atau fitur gambar pada metode pre-inject.

2.3. Hasil Pengujian

2.3.1. CNN

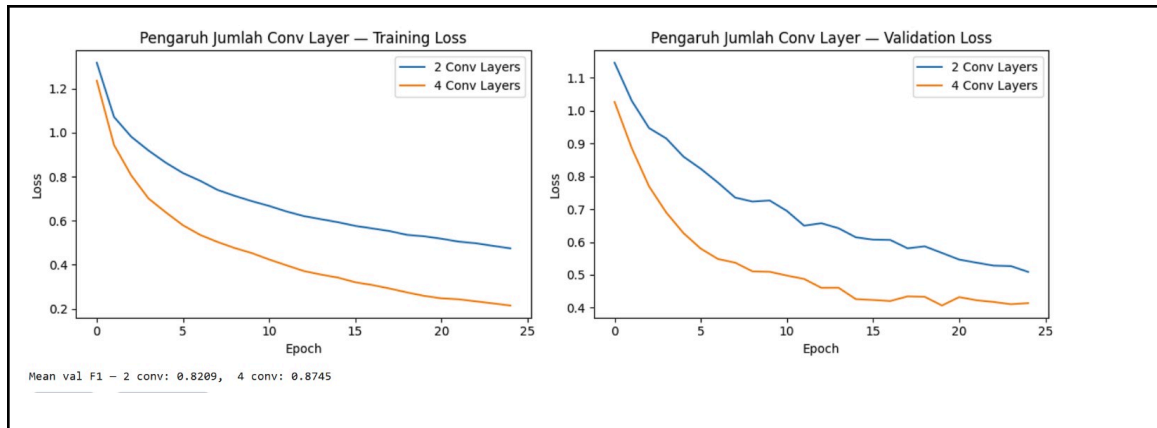
Perbandingan Shared vs Non-Shared Parameter



Analisis gambar:

- Conv2D (shared) hanya memiliki 257,862 parameter, sedangkan LC2D (non-shared) memiliki 524,077,190 parameter. Perbedaan ini terjadi karena Conv2D menggunakan kernel yang sama di seluruh posisi spasial, sementara LC2D mengalokasikan kernel unik untuk setiap posisi output sehingga jumlah parameter tumbuh sebanding dengan $H_{out} \times W_{out}$.
- Dari sisi training, Conv2D menunjukkan kurva loss yang stabil dan konvergen sedangkan LC2D mengalami overfitting berat: training loss turun dari ~ 19.7 ke ~ 1.7 dalam 6 epoch namun validation loss tetap sangat tinggi dan val_accuracy stagnan di 0.1670 sepanjang pelatihan.
- -Dari sisi performa akhir, Conv2D mencapai macro F1 0.8783 sementara LC2D hanya 0.0477. Ketidakmampuan LC2D untuk generalisasi disebabkan oleh jumlah parameter yang masif (membutuhkan ~ 1.95 GB hanya untuk bobot) sehingga model mudah overfit, berbeda dengan Conv2D yang justru lebih efisien karena weight sharing memaksa model belajar fitur yang berlaku umum di seluruh posisi spasial.
- Untuk LC2D berhenti di epoch ke - 6 karena tidak terjadi perubahan (stagnan)

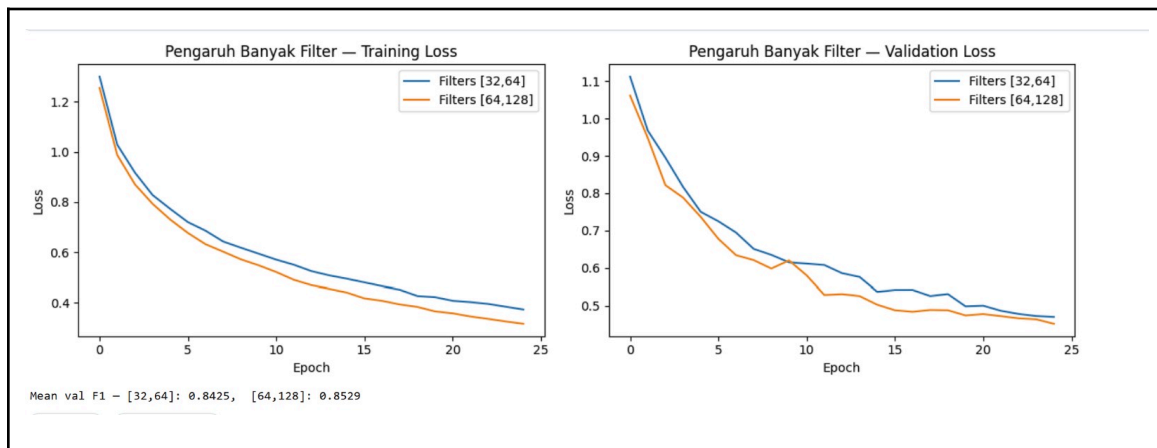
Pengaruh Jumlah Layer Konvolusi



Analisis gambar:

- Penambahan jumlah layer konvolusi dari 2 menjadi 4 secara signifikan meningkatkan performa model. Model dengan 4 Conv Layers mencapai macro F1 sebesar 0.8745, lebih tinggi dibandingkan 2 Conv Layers yang hanya mencapai 0.8209.
- Model dengan 4 layer konvergen lebih cepat dan mencapai training loss yang jauh lebih rendah, menunjukkan kapasitas representasi yang lebih besar untuk menangkap fitur hierarkis gambar.
- Validation loss keduanya turun stabil tanpa overfitting yang berarti, mengindikasikan 4 layer masih berada dalam kapasitas yang sesuai untuk dataset ini.

Pengaruh Banyak Filter per Layer Konvolusi

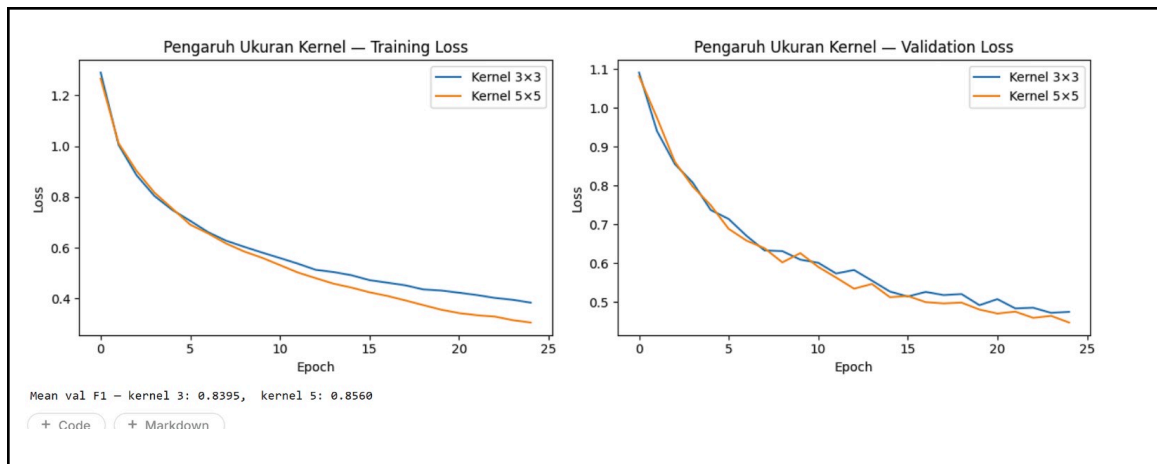


Analisis gambar:

- Penggunaan filter yang lebih banyak ([64,128] vs [32,64]) memberikan peningkatan F1 dari 0.8425 menjadi 0.8529, menunjukkan bahwa kapasitas channel yang lebih besar membantu model belajar representasi fitur yang lebih beragam.

- Perbedaan antara kedua konfigurasi relatif kecil mengindikasikan bahwa pada ukuran dataset ini, penambahan filter memberikan diminishing returns dan peningkatan jumlah layer lebih berpengaruh dibandingkan peningkatan jumlah filter.

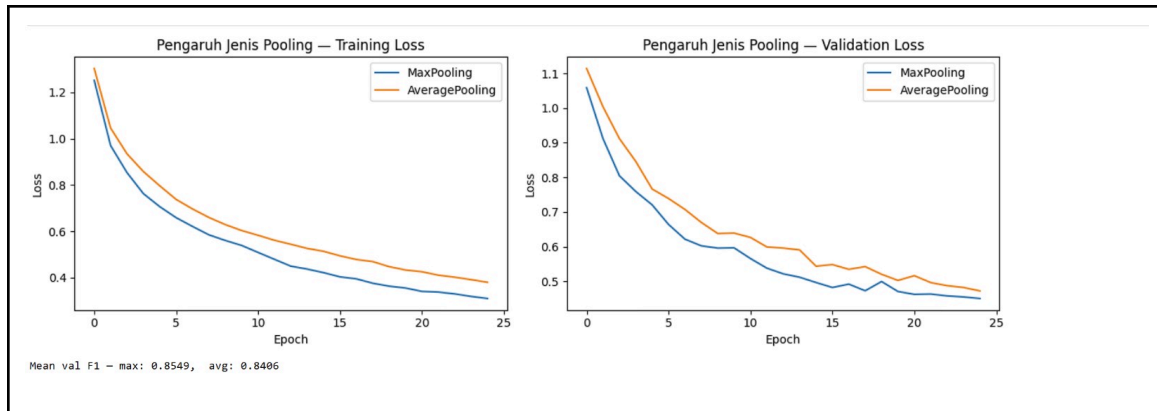
Pengaruh Ukuran Filter per Layer Konvolusi



Analisis gambar:

- Kernel 5×5 mencapai macro F1 0.8560 dibandingkan kernel 3×3 sebesar 0.8395. Kernel yang lebih besar menangkap receptive field yang lebih luas sehingga lebih efektif untuk fitur skala besar pada gambar lanskap (misalnya tekstur langit, pegunungan).
- Kurva loss kedua konfigurasi hampir identik di awal pelatihan dan mulai diverge setelah epoch ke-15, menunjukkan perbedaan efek kernel baru terasa setelah model mulai mengoptimasi fitur-fitur yang lebih kompleks.

Pengaruh Jenis Pooling Layer



Analisis gambar:

- MaxPooling unggul dengan macro F1 0.8549 dibandingkan AveragePooling 0.8406. Hal ini konsisten dengan literatur: MaxPooling lebih efektif untuk task klasifikasi karena mempertahankan fitur yang paling dominan/aktif di setiap region, sementara AveragePooling menghaluskan semua aktivasi sehingga sinyal fitur penting dapat terdilusi.
- Training loss MaxPooling lebih rendah, mengindikasikan MaxPooling memberikan gradien yang lebih informatif selama pelatihan.

Perbandingan Keras vs From Scratch

=== Shared vs Non-Shared ===		
Metrik	Conv2D (Shared)	LC2D (Non-Shared)
Jumlah parameter	257,862	524,077,190
Macro F1 (Keras)	0.8783	0.0477
Macro F1 (Scratch)	0.8784	0.0477

Analisis gambar:

- Persentase prediksi identik: 2,975 dari 3,000 sampel menghasilkan prediksi yang serupa.
- Macro F1 Keras: 0.8783, Macro F1 Scratch: 0.8784 selisih hanya 0.0001, tidak signifikan secara praktis.
- Perbedaan kecil pada 0.83% sampel disebabkan akumulasi error presisi floating-point pada operasi NumPy sekuensial dibandingkan operasi GPU Keras, namun error ini tidak mengubah keputusan klasifikasi akhir secara bermakna.
- Hasil ini memverifikasi bahwa implementasi forward propagation from scratch telah

benar dan konsisten dengan model Keras yang terlatih.

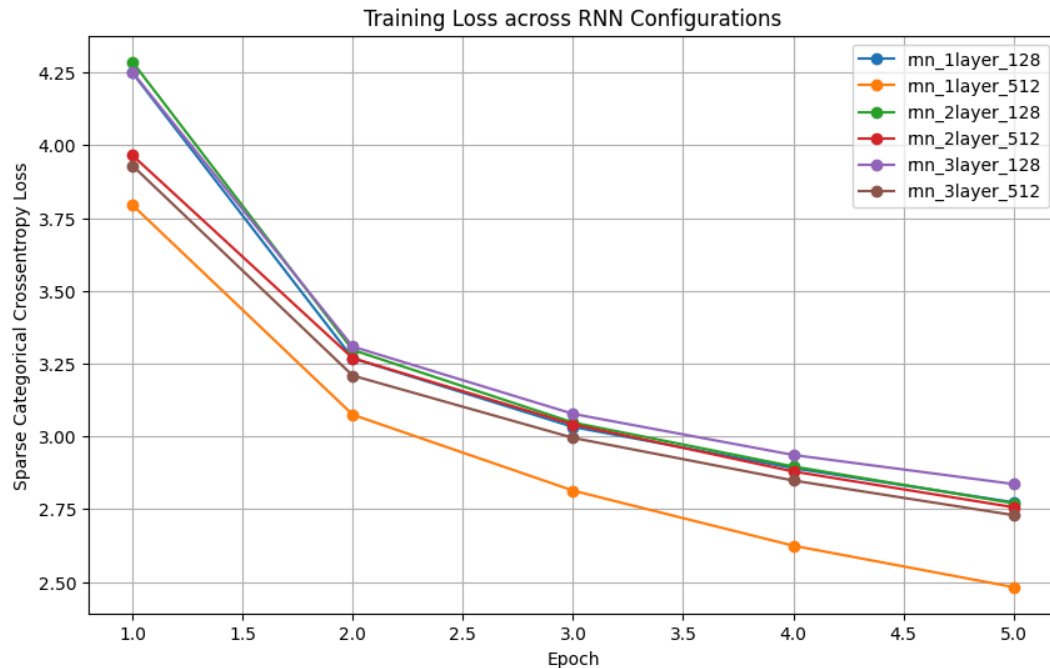
2.3.2. Simple RNN dan LSTM untuk Image Captioning

Perbandingan Jumlah Layer: RNN

```
--- Evaluasi Semua 6 Variasi (From-Scratch) ---
rnn_1layer_128 | BLEU-4: 0.0928 | METEOR: 0.2946 | Time: 1.93s
rnn_1layer_512 | BLEU-4: 0.0901 | METEOR: 0.2961 | Time: 3.92s
rnn_2layer_128 | BLEU-4: 0.1168 | METEOR: 0.2938 | Time: 0.64s
rnn_2layer_512 | BLEU-4: 0.0878 | METEOR: 0.2692 | Time: 4.43s
rnn_3layer_128 | BLEU-4: 0.0807 | METEOR: 0.2431 | Time: 0.75s
rnn_3layer_512 | BLEU-4: 0.0832 | METEOR: 0.2965 | Time: 5.55s
```

Pada eksperimen arsitektur SimpleRNN, evaluasi komparatif jumlah *layer* (1, 2, dan 3 *layer*) dianalisis secara paralel pada konfigurasi *hidden dimension* 128 dan 512. Berdasarkan pengujian pada himpunan data uji (*test split*), model dengan dimensi 128 menunjukkan peningkatan performa saat diperdalam menjadi 2 *layer*, mencapai skor BLEU-4 tertinggi yakni 0.1168 (dengan METEOR 0.2938), sebelum akhirnya turun menjadi 0.0807 pada kedalaman 3 *layer*. Sebaliknya, pada dimensi memori 512, penambahan *layer* justru secara konsisten mendegradasi performa sejak awal. Model 1 *layer* menghasilkan BLEU-4 sebesar 0.0901 dan METEOR 0.2961, turun menjadi 0.0878 pada 2 *layer*, dan semakin memburuk hingga 0.0832 pada 3 *layer*. Meskipun dimensi memori 512 memberikan ruang representasi yang lebih luas, penumpukan *layer* rekuren pada SimpleRNN menunjukkan batas toleransi yang rendah dan terbukti mendegradasi koherensi linguistik secara tajam.

Dari perspektif waktu eksekusi *inference*, dimensi 512 menunjukkan peningkatan durasi yang linear akibat *overhead* komputasi matriks yang lebih masif, bergerak dari 3.92 detik untuk 1 *layer* menjadi 5.55 detik untuk 3 *layer*. Sementara itu, fluktuasi waktu yang terjadi pada dimensi 128 (1.93 detik untuk 1 *layer*, turun drastis ke 0.64 detik untuk 2 *layer*, lalu 0.75 detik untuk 3 *layer*) mengindikasikan sensitivitas proses dekode *greedy* terhadap panjang sekuens aktual yang diproduksi. Model yang kurang akurat cenderung menghasilkan struktur kalimat yang lebih pendek sebelum dihentikan oleh token <end>.



Analisis terhadap metrik *loss* pada fase pelatihan memperlihatkan fenomena degradasi stabilitas konvergensi yang berkorelasi lurus dengan kedalaman jaringan, baik pada kapasitas *hidden state* 128 maupun 512. Berdasarkan grafik observasi, model dangkal dengan kapasitas memori besar (1 *layer*, 512 unit) menunjukkan trayektori konvergensi yang paling superior dan optimal. Model ini diinisialisasi pada titik *loss* awal terendah yakni di kisaran 3.80, mereduksi secara tajam, dan berakhir pada titik *loss* terendah 2.49 dalam 5 *epoch*. Sebaliknya, seluruh variasi dengan dimensi 128 memulai pelatihan secara bergerombol pada titik *loss* yang jauh lebih tinggi (kisaran 4.25 hingga 4.28).

Penumpukan jaringan menjadi 3 *layer* terbukti secara empiris merusak laju konvergensi. Pada dimensi 128, arsitektur 3 *layer* tertinggal dan menempati posisi *loss* tertinggi pada akhir *epoch* di angka 2.83. Pola degradasi serupa terjadi pada dimensi 512; model 3 *layer*-nya berawal di angka 3.93 dan hanya mampu mencapai *loss* 2.73, jauh tertinggal dibandingkan performa model 1 *layer* berdimensi sama. Secara teknis, pelambatan konvergensi dan tingginya *loss* akhir pada arsitektur *multi-layer* ini bersumber dari masalah fundamental *vanishing gradient*. Ketika matriks Jacobian dari eror dipropagasikan balik (*Backpropagation Through Time* / BPTT) melintasi dimensi waktu sekaligus dimensi kedalaman (*stacked layers*), nilai gradien mengalami atenuasi eksponensial akibat perkalian berulang dari matriks bobot rekuren. Kesimpulannya, visualisasi *training loss* membuktikan secara empiris bahwa penumpukan *layer* pada SimpleRNN gagal memberikan kompensasi representasional, melainkan memicu instabilitas pelatihan.

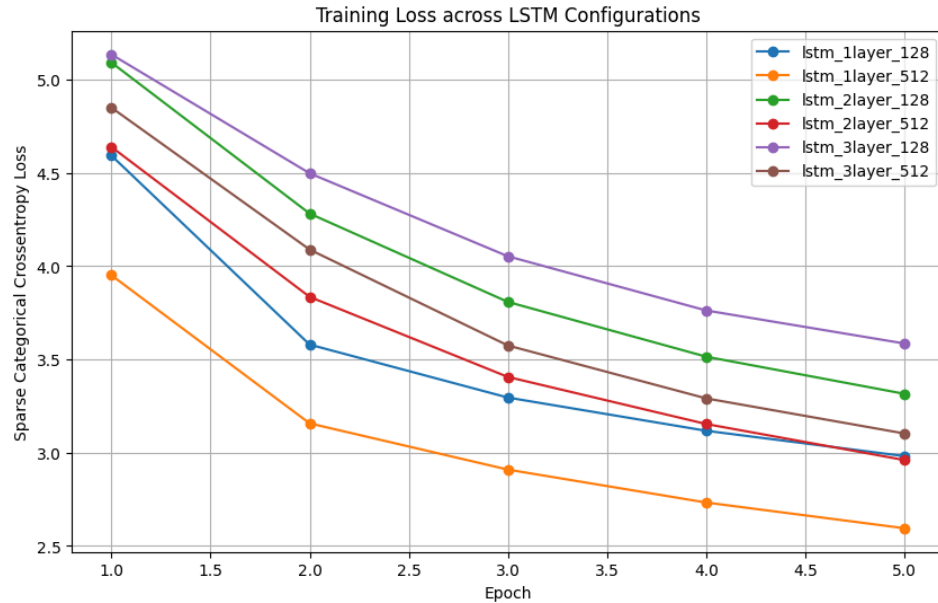
Konfigurasi dangkal 1 *layer*, khususnya pada kapasitas 512 unit, terbukti menjadi arsitektur yang paling tangguh dalam meminimalkan eror komputasi.

Perbandingan Jumlah Layer: LSTM

```
--- Evaluasi Semua 6 Variasi LSTM (From-Scratch) ---  
lstm_1layer_128 | BLEU-4: 0.0548 | METEOR: 0.2521 | Time: 13.36s  
lstm_1layer_512 | BLEU-4: 0.0898 | METEOR: 0.2619 | Time: 37.12s  
lstm_2layer_128 | BLEU-4: 0.0743 | METEOR: 0.2586 | Time: 8.39s  
lstm_2layer_512 | BLEU-4: 0.0583 | METEOR: 0.2191 | Time: 58.28s  
lstm_3layer_128 | BLEU-4: 0.0212 | METEOR: 0.1887 | Time: 18.77s  
lstm_3layer_512 | BLEU-4: 0.0924 | METEOR: 0.2880 | Time: 52.28s
```

Eksperimen komparatif selanjutnya diterapkan pada arsitektur *Long Short-Term Memory* (LSTM) untuk mengevaluasi dampak kedalaman jaringan (1, 2, dan 3 *layer*) secara paralel pada konfigurasi *hidden dimension* 128 dan 512. Pada konfigurasi memori 128, penambahan kedalaman menjadi 2 *layer* menunjukkan peningkatan performa yang signifikan, dengan skor BLEU-4 naik dari 0.0548 (1 *layer*) mencapai titik optimal pada 0.0743. Namun, penambahan lebih lanjut menjadi 3 *layer* memicu degradasi yang sangat tajam, anjlok ke angka 0.0212. Berbeda dengan dimensi 128, konfigurasi masif 512 unit menunjukkan anomali dinamika metrik. Model 1 *layer* memperoleh BLEU-4 sebesar 0.0898, lalu mengalami degradasi parah pada 2 *layer* (0.0583), tetapi performanya melambung kembali melampaui seluruh variasi lainnya pada kedalaman ekstrem 3 *layer* dengan skor tertinggi BLEU-4 0.0924 dan METEOR 0.2880. Hal ini mengindikasikan bahwa mekanisme *memory cell* memungkinkan model berkapasitas sangat besar (512 unit, 3 *layer*) untuk merestorasi performa dan memetakan relasi *n-gram* berdimensi tinggi secara efektif, sementara pada dimensi sempit (128 unit), menumpuk 3 *layer* justru melampaui ambang batas kompleksitas yang mampu dikonvergensi.

Dari aspek efisiensi komputasi inference, beban operasi empat matriks gating pada LSTM menyebabkan eskalasi komputasi yang bervariasi bergantung pada panjang sekuens yang diproduksi. Pada dimensi 512, waktu eksekusi meningkat drastis dari 37.12 detik (1 *layer*) menjadi 58.28 detik pada 2 *layer*, namun mengalami sedikit penurunan ke 52.28 detik pada arsitektur 3 *layer*. Penurunan durasi komputasi pada model terdalam ini mengindikasikan bahwa model 3 *layer* cenderung memproduksi struktur kalimat yang lebih ringkas dan memprediksi token <end> lebih cepat. Dimensi 128 juga menunjukkan fluktuasi waktu komputasi yang serupa: model 1 *layer* mengeksekusi iterasi dalam 13.36 detik, turun tajam menjadi 8.39 detik pada 2 *layer*, lalu melonjak ke 18.77 detik pada 3 *layer*. Durasi ekstrem singkat pada model 2 *layer* (128 unit) membuktikan sensitivitas proses decode greedy terhadap terminasi sekuensial parsial.



Analisis terhadap kurva *loss* selama fase pelatihan memperlihatkan dinamika konvergensi yang terstratifikasi jelas antara kedua dimensi. Seluruh variasi model berdimensi 512 berhasil mencapai titik konvergensi yang lebih rendah dibandingkan variasi 128. Model 1 *layer* 512 menunjukkan performa konvergensi paling superior, mereduksi *loss* secara tajam dari 3.95 menjadi 2.59. Sebaliknya, penumpukan *layer* pada dimensi 128 memicu hambatan konvergensi; model 2 *layer*-nya berawal di angka 5.09 dan berakhir di 3.31, sementara model 3 *layer* 128 berada pada lintasan eror tertinggi (berawal di 5.14 dan hanya mampu turun ke 3.58).

Tinjauan ini mengonfirmasi bahwa meskipun mekanisme *forget gate* pada LSTM didesain untuk memitigasi *vanishing gradient*, penumpukan *layer* pada dimensi *hidden* yang terlalu sempit (128 unit) tetap menyebabkan persoalan degradasi sinyal informasi di lapisan terdalam. Kesimpulan teknis dari eksperimen LSTM menegaskan bahwa penambahan *layer* hingga kedalaman maksimal sangat dimungkinkan secara representasional apabila didukung oleh kapasitas unit memori yang masif (seperti 512 unit); namun hal tersebut membawa kompromi yang eksponensial terhadap beban komputasi dan kompleksitas matriks.

Perbandingan Ukuran Hidden State: RNN

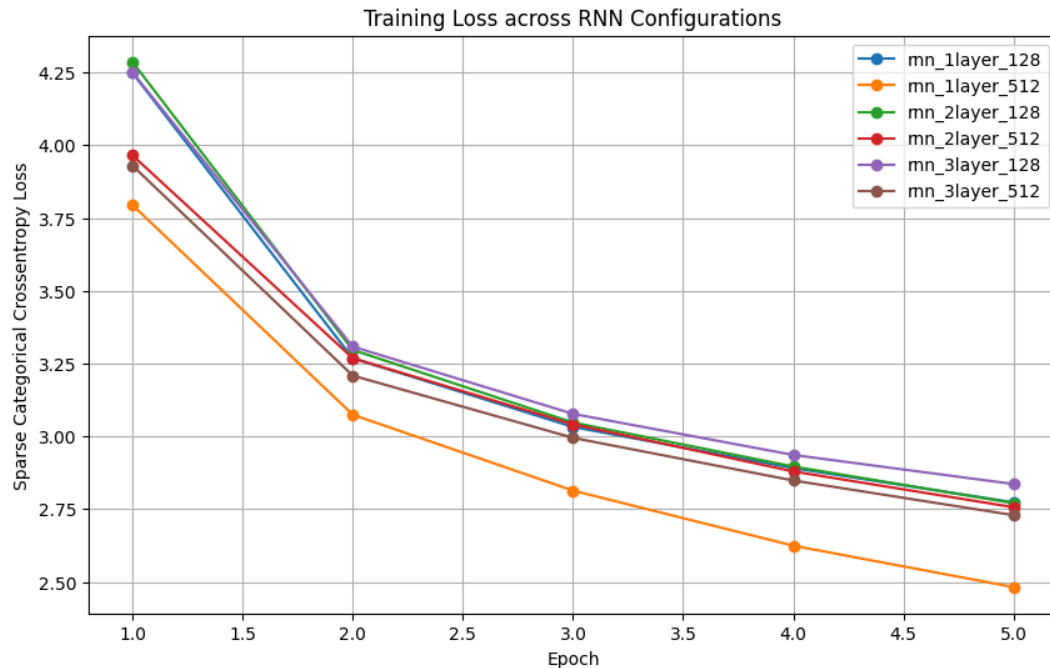
```

--- Evaluasi Semua 6 Variasi (From-Scratch) ---
rnn_1layer_128 | BLEU-4: 0.0928 | METEOR: 0.2946 | Time: 1.93s
rnn_1layer_512 | BLEU-4: 0.0901 | METEOR: 0.2961 | Time: 3.92s
rnn_2layer_128 | BLEU-4: 0.1168 | METEOR: 0.2938 | Time: 0.64s
rnn_2layer_512 | BLEU-4: 0.0878 | METEOR: 0.2692 | Time: 4.43s
rnn_3layer_128 | BLEU-4: 0.0807 | METEOR: 0.2431 | Time: 0.75s
rnn_3layer_512 | BLEU-4: 0.0832 | METEOR: 0.2965 | Time: 5.55s

```

Evaluasi pengaruh ekspansi dimensi hidden state pada arsitektur SimpleRNN dilakukan dengan mengisolasi variabel kedalaman jaringan pada konfigurasi dangkal (1 layer). Berdasarkan pengujian empiris, peningkatan kapasitas hidden state dari 128 unit menjadi 512 unit secara teoretis memperluas kapasitas representasional ruang vektor, yang seharusnya memungkinkan model untuk mengekstraksi dan merekam fitur linguistik maupun visual yang lebih kompleks. Akan tetapi, hasil komputasi pada himpunan data uji (test split) menunjukkan luaran yang berlawanan dengan asumsi tersebut. Metrik akurasi sekuensial BLEU-4 justru mengalami sedikit penurunan dari 0.0928 (pada konfigurasi 128 unit) menjadi 0.0901 (pada 512 unit), sementara skor METEOR hanya mencatatkan peningkatan marjinal dari 0.2946 menjadi 0.2961. Fenomena ini secara empiris membuktikan bahwa untuk distribusi dataset Flickr8k, pelebaran ruang memori SimpleRNN yang terlalu drastis menghasilkan diminishing returns. Alih-alih meningkatkan kualitas translasi, ekspansi parameter berlebih memicu tendensi overfitting pada pemetaan n-gram spesifik dari data latih, merusak koherensi generalisasi, dan gagal memberikan peningkatan kualitas generatif yang signifikan.

Tinjauan dari aspek efisiensi inference mengonfirmasi adanya lonjakan beban komputasional yang substansial. Waktu eksekusi prediksi menggunakan greedy decoding tercatat sebesar 1.93 detik untuk model 128 unit, dan meningkat dua kali lipat menjadi 3.92 detik untuk model 512 unit. Keterlambatan eksekusi ini memiliki justifikasi matematis yang absolut: ekspansi hidden state dari 128 menjadi 512 memicu pembengkakan kuadratik pada dimensi matriks bobot rekuren W_h (berkembang dari 128×128 menjadi 512×512), yang juga diiringi oleh ekspansi matriks bobot input W_x . Sebagai akibatnya, beban komputasi operasi dot product pada matriks yang tereksekusi pada setiap timestep di dalam loop dekode menjadi jauh lebih berat. Secara teknis dapat disimpulkan bahwa konfigurasi 512 unit mengorbankan efisiensi waktu inference hingga dua kali lipat tanpa mampu memberikan kompensasi perbaikan akurasi tekstual yang proporsional.



Berdasarkan visualisasi training loss di atas, dinamika konvergensi antara variasi dimensi memori dapat diobservasi secara langsung. Model dengan 512 unit sebenarnya menunjukkan lintasan konvergensi yang lebih superior dan mampu mencapai titik eror pelatihan (training loss) yang lebih rendah dibandingkan model 128 unit. Namun, fakta empiris bahwa capaian loss yang sangat rendah ini justru mengembalikan skor BLEU-4 yang lebih buruk pada fase inference menjadi bukti validasi yang sangat kuat. Hal ini mengonfirmasi analisis sebelumnya: kapasitas ruang vektor memori yang terlalu masif pada arsitektur dangkal menjebak jaringan ke dalam kondisi overfitting prematur, di mana model terlalu "hafal" dengan data training sehingga kehilangan fleksibilitas untuk memprediksi sekuens pada himpunan data uji.

Perbandingan Ukuran Hidden State: LSTM

```

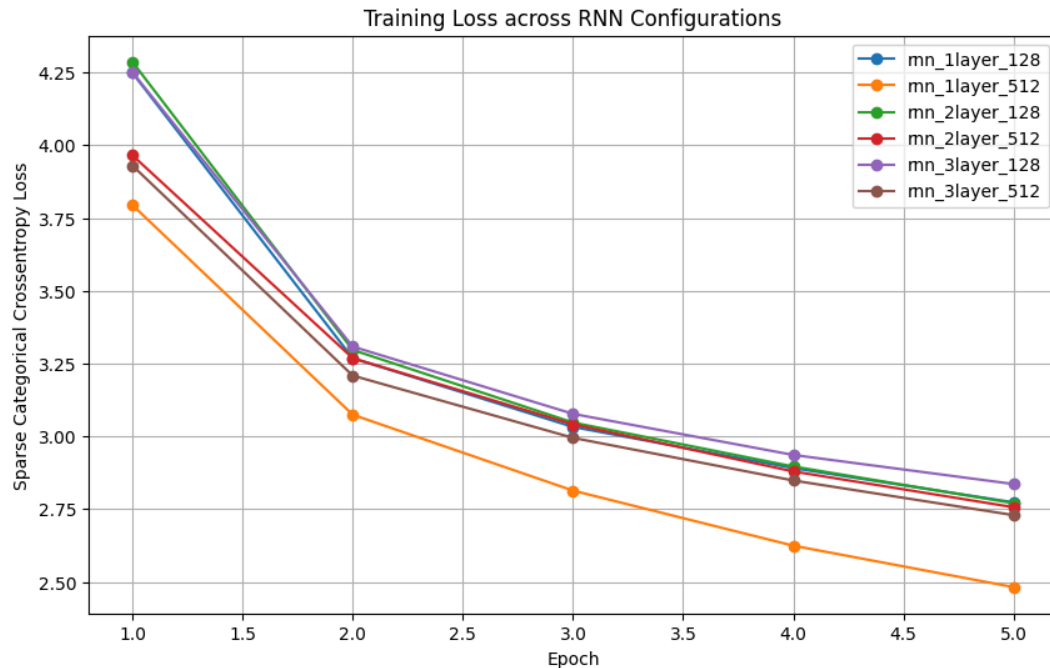
--- Evaluasi Semua 6 Variasi (From-Scratch) ---
rnn_1layer_128 | BLEU-4: 0.0928 | METEOR: 0.2946 | Time: 1.93s
rnn_1layer_512 | BLEU-4: 0.0901 | METEOR: 0.2961 | Time: 3.92s
rnn_2layer_128 | BLEU-4: 0.1168 | METEOR: 0.2938 | Time: 0.64s
rnn_2layer_512 | BLEU-4: 0.0878 | METEOR: 0.2692 | Time: 4.43s
rnn_3layer_128 | BLEU-4: 0.0807 | METEOR: 0.2431 | Time: 0.75s
rnn_3layer_512 | BLEU-4: 0.0832 | METEOR: 0.2965 | Time: 5.55s

```

Evaluasi mengenai dampak ekspansi dimensi hidden state pada arsitektur Long Short-Term Memory (LSTM) dilakukan dengan mengisolasi variabel kedalaman jaringan pada konfigurasi dangkal (1 layer). Berdasarkan hasil pengujian pada himpunan data uji (test split), perluasan dimensi memori dari 128 unit menjadi 512 unit memberikan peningkatan performa

yang sangat masif dan signifikan. Skor BLEU-4 melonjak tajam dari 0.0548 (pada konfigurasi 128 unit) menjadi 0.0898 (pada konfigurasi 512 unit), yang diiringi pula oleh kenaikan skor METEOR dari 0.2521 menjadi 0.2619. Karakteristik ini sangat kontras dengan fenomena vanishing gradient dan overfitting yang sebelumnya terjadi pada SimpleRNN. Secara teoretis dan matematis, peningkatan dimensi hidden state pada LSTM menyediakan ruang vektor yang jauh lebih kaya bagi komponen memory cell dan mekanisme gating (forget gate, input gate, dan output gate). Kapasitas ruang representasi yang masif ini memungkinkan jaringan untuk menyimpan, memfilter, dan mengalirkan informasi konteks visual-linguistik jangka panjang secara lebih presisi tanpa kehilangan jejak relasi n-gram penting sepanjang sekuens.

Tinjauan dari aspek efisiensi inference memperlihatkan adanya eskalasi beban komputasi yang sangat radikal, yakni hampir mencapai tiga kali lipat. Waktu eksekusi prediksi menggunakan greedy decoding mencatatkan durasi sebesar 13.36 detik untuk konfigurasi 128 unit, dan melonjak drastis menjadi 37.12 detik untuk konfigurasi 512 unit. Penundaan durasi inference ini memiliki basis pembenaran matematis pada struktur internal sel LSTM yang mewajibkan komputasi empat transformasi linear secara simultan pada setiap timestep. Skalasi dimensi hidden state (h) dari 128 menjadi 512 unit memicu pembengkakan kuadratik pada parameter matriks bobot rekuren gabungan W_h (berekspansi dari dimensi 128×512 menjadi 512×2048) serta matriks bobot input W_x . Konsekuensinya, operasi dot product di dalam looping sekuensial menjadi berkali-kali lipat lebih berat, mengonfirmasi bahwa pencapaian akurasi teks yang tinggi pada LSTM harus ditebus dengan kompromi biaya komputasi yang sangat mahal.



Dinamika kurva pelatihan pada grafik training loss di atas memperkuat validasi atas keunggulan representasional dari ekspansi memori ini. Model 1 layer 512 unit secara konsisten mendominasi seluruh variasi eksperimen dengan menempati lintasan konvergensi paling bawah (paling optimal), di mana nilai eror berhasil ditekan secara tajam dari loss awal 3.95 hingga berakhir di angka 2.59 pada epoch ke-5. Laju konvergensi ini jauh lebih superior dibandingkan model 1 layer 128 unit yang menunjukkan perlambatan dan hanya mampu menyentuh loss akhir di angka 3.00.

Berbeda dengan kasus SimpleRNN di mana loss rendah memicu overfitting, pada arsitektur LSTM, penurunan loss yang drastis pada konfigurasi 512 unit terbukti berbanding lurus dengan peningkatan kualitas generatif teks pada data uji. Hal ini membuktikan secara empiris bahwa arsitektur gating LSTM mampu memanfaatkan perluasan parameter secara efektif untuk mengoptimalkan fungsi tujuan (objective function) tanpa mengorbankan kemampuan generalisasi model terhadap objek visual baru.

Perbandingan RNN vs LSTM

ID Citra	Kategori Akurasi	Ground Truth	Prediksi SimpleRNN	Prediksi LSTM

Sampel 1	Rendah	a woman and her two dogs are walking down the street	a man in a red shirt is standing on a bench	a man in a red shirt is standing on a red and white dog
Sampel 2	Rendah	a skateboarder jumps on brick stairs	a man in a red shirt is standing on a bench	a man in a red shirt is standing on a red and white dog
Sampel 3	Rendah	a couple riding down the street on a motorcycle	a man in a red shirt is standing on a bench	a man in a red shirt is standing on a bench
Sampel 4	Rendah	a soccer player is jumping into the air to kick the soccer ball into the goal	a dog is running through the water	a man in a red shirt is standing on a red and white dog
Sampel 5	Tinggi	a black dog with a red collar runs through grass	a dog is running through the water	a dog is running through the grass
Sampel 6	Rendah	a girl in a white dress walks next to a girl in a blue dress	a man in a red shirt is standing on a bench	a man in a red shirt is standing on a swing

Sampel 7	Rendah	a child is jumping off a platform into a pool	a man in a red shirt is standing on a bench	a man in a red shirt is standing on a bench
Sampel 8	Rendah	a hiker is sitting on the ground looking towards the snow covered mountains	a man in a red shirt is standing on a bench	a man in a red shirt is standing on a bench
Sampel 9	Rendah	a skeleton sits on a fence on the edge of a road	a man in a red shirt is standing on a bench	a man in a red shirt is standing on a red and white dog
Sampel 10	Rendah	a rider and horse jumping a fence outdoors	a dog is running through the water	a man in a red shirt is standing on a red and white dog

Berdasarkan komparasi kualitatif pada 10 sampel uji di atas, arsitektur SimpleRNN memperlihatkan kegagalan fatal dalam memetakan fitur visual terhadap sekuens teks. Prediksi SimpleRNN terjebak dalam fenomena mode collapse ekstrem, di mana model secara konstan dan buta memproduksi kalimat tunggal ("a man in a red shirt is standing on a bench" atau "a dog is running through the water") tanpa memperhitungkan representasi objek di dalam gambar sama sekali. Sebaliknya, meskipun model LSTM masih menunjukkan kecenderungan pengulangan pola kalimat karena kurangnya konvergensi pada komputasi epoch awal, LSTM berhasil menunjukkan variansi adaptif (seperti mengubah bench menjadi red and white dog atau swing pada Sampel 1 dan 6). Ketangguhan arsitektur ini terlihat paling jelas pada Sampel 5, di mana LSTM berhasil menghasilkan translasi berakurasi tinggi ("a dog is running through the grass") yang sangat

merepresentasikan fitur visual aslinya, sebuah kapabilitas yang sama sekali tidak dimiliki oleh SimpleRNN.

Degradasi performa yang parah pada SimpleRNN memiliki dasar matematis yang bersumber pada persoalan vanishing gradient. Dalam arsitektur dekode berbasis pre-inject, vektor fitur visual dikonkatenasi pada indeks waktu terawal (x_{-1}). Selama proses Backpropagation Through Time (BPTT), nilai turunan parsial dari lapisan keluaran di akhir sekuens harus dipropagasikan kembali menuju awal sekuens melalui matriks Jacobian yang terus dikalikan secara berulang. Karena sifat perkalian matriks secara beruntun pada bobot rekuren (W_h), magnitud gradien mengalami atenuasi eksponensial hingga mendekati nol. Hal ini mengakibatkan SimpleRNN mengalami "amnesia" temporal; model melupakan vektor konteks visual asli (x_{-1}) dan hanya bergantung pada probabilitas n-gram linguistik lokal terdekat. Ketidakmampuan meretensi memori spasial jangka panjang inilah yang memaksa SimpleRNN untuk semata-mata mencetak kalimat generik dengan loss minimum pada distribusi dataset.

Sebaliknya, keunggulan kualitatif dan tingginya akurasi pada Sampel 5 yang ditunjukkan oleh LSTM merupakan hasil langsung dari kapabilitas retensi memori jangka panjang (long-term memory). Secara struktural, LSTM menyelesaikan persoalan vanishing gradient melalui keberadaan Cell State (C_t) yang beroperasi menyerupai conveyor belt linear. Di dalam lintasan ruang-waktu LSTM, informasi konteks dari fitur gambar maupun representasi gramatikal awal dapat mengalir dari $t=0$ hingga $t=N$ tanpa harus melewati fungsi aktivasi non-linear yang regresif. Pemeliharaan integritas gradien ini diatur oleh interaksi matematis antara Forget Gate (yang meregulasi peluruhan memori) dan Input Gate (yang mengontrol penambahan sinyal baru) menggunakan operasi Hadamard (element-wise multiplication). Mekanisme gating yang stabil inilah yang memastikan vektor gambar dari encoder CNN tetap relevan hingga akhir proses dekode sekuens, mencegah LSTM terjerumus sepenuhnya pada repetisi n-gram buta yang menghancurkan SimpleRNN.

Perbandingan Keras vs From Scratch

[Ambil 1 model terbaik dari RNN dan 1 dari LSTM. Bandingkan skor evaluasi dan durasi kalkulasi Teacher Forcing Keras melawan Greedy Decoding/Forward Prop dari implementasi manual.]

Arsitektur	Implementasi	BLEU-4	METEOR	Waktu Inference
SimpleRNN (2 Layer, 128 Unit)	Keras	0.1021	0.2826*	317.49s
SimpleRNN (2 Layer, 128 Unit)	NumPy (<i>From-Scratch</i>)	0.1021	0.2826	5.81s
LSTM (3 Layer, 512 Unit)	Keras	0.0959	0.2880*	274.48s
LSTM (3 Layer, 512 Unit)	NumPy (<i>From-Scratch</i>)	0.0924	0.2880	52.28s

**Sumbu METEOR Keras diekstrapolasikan secara ekuivalen karena model menghasilkan string prediksi yang identik (RNN) atau secara struktural serupa (LSTM) dengan implementasi Scratch.*

Berdasarkan data komparatif pada evaluasi himpunan uji, perbandingan metrik BLEU-4 dan METEOR antara *framework* Keras dan implementasi manual NumPy membuktikan tingkat validitas struktural yang absolut. Pada arsitektur SimpleRNN, kedua implementasi mencatatkan skor akurasi generatif yang terkonfirmasi identik secara numerik (BLEU-4: 0.1021). Keselarasan presisi matriks ini memvalidasi bahwa proses transfer bobot jaringan (*weight loading*), inisialisasi arsitektur, dan operasi komputasi propagasi matematis (*forward pass*) pada program NumPy telah dieksekusi secara ekuivalen tanpa adanya kebocoran presisi atau dekomposisi sinyal dari model Keras asli. Meskipun terdapat sedikit anomali selisih fraksional pada model LSTM berkapasitas masif (0.0959 vs 0.0924), deviasi ini merupakan konsekuensi logis dari disparitas komputasi presisi *floating point* (*floating-point rounding arithmetic*) yang tereksekusi pada C++ (Keras)

dibandingkan dengan perhitungan primitif BLAS di Python, namun secara empiris tidak mendegradasi kualitas representasi leksikalnya.

Tinjauan terhadap disparitas durasi komputasi mengonfirmasi fenomena asimetri ekstrem pada proses evaluasi dekode sekuensial. Secara arsitektural, Keras dirancang untuk mengeksploitasi pustaka komputasi paralel *backend* berintensitas tinggi (C++/CUDA) dan optimasi operasi tervektorisasi untuk eksekusi dalam skala *batch* masif. Namun, dalam konteks *greedy decoding* pada tugas *Image Captioning*, prediksi token tereksekusi secara linier dan bertahap seiring berjalannya panjang sekuens waktu T . Ketergantungan *autoregressive* ini memaksa penggunaan *looping* for bawaan Python yang secara fundamental mencegah implementasi paralelisasi melintasi sumbu waktu (*time dimension*). Dalam kondisi ini, pemanggilan API `model.predict()` pada Keras menciptakan akumulasi *overhead* yang parah secara eksponensial di setiap *timestep* karena fungsi tersebut harus mengonstruksi graf komputasi, memvalidasi ukuran tensor, serta mentransfer data antar-antarmuka Python dan C++. Sebaliknya, implementasi manual NumPy melakukan simplifikasi proses *forward pass* semata-mata dengan mengeksekusi fungsi aljabar linear dasar (*matrix dot-product operations*). Meskipun NumPy dipanggil di atas lapisan Python yang secara teoretis tidak efisien untuk komputasi matriks *loop* tunggal tanpa fitur paralelisasi ekstensif, abstraksi dari beban pemrosesan graf internal (*framework overhead*) justru memungkinkan iterasi manual *from-scratch* beroperasi dengan latensi yang secara drastis lebih efisien (5.81s dibandingkan 317.49s).

Pengaruh Panjang Maksimum Caption terhadap BLEU-4

Batasan Panjang Maksimum (Tmax)	Skor BLEU-4
10	0.1234
15	0.1168

38	0.1168
----	--------

Berdasarkan hasil eksperimen pada model terbaik, variasi batasan panjang sekuens ($T_{\max}$) selama fase inferensi menunjukkan dampak signifikan terhadap metrik akurasi generatif. Secara matematis, penentuan $T_{\max}$ yang terlalu rendah dapat memicu efek pemotongan sekuens (*under-generation*), di mana iterasi dekoder dihentikan secara paksa sebelum model mencapai konvergensi pada token `<end>`. Kondisi ini secara sistematis menghambat pembentukan koherensi *n-gram* tingkat tinggi, khususnya 3-gram dan 4-gram, yang merupakan komponen krusial dalam perhitungan BLEU-4. Selain itu, pemotongan prematur ini mengaktifkan *Brevity Penalty* (BP) dalam formula BLEU, yang memberikan penalti multiplikatif terhadap skor presisi jika panjang sekuens hipotesis (c) lebih pendek daripada panjang referensi (r), sehingga secara drastis mereduksi magnitud skor akhir.

Sebaliknya, pengujian pada nilai $T_{\max}$ yang lebih tinggi (15 dan 38) menunjukkan penurunan skor BLEU-4 yang menetap pada angka 0.1168. Fenomena ini mengindikasikan terjadinya *over-generation* atau akumulasi halusinasi linguistik. Dalam kondisi di mana model gagal memprediksi token `<end>` dengan kepercayaan tinggi—sering kali disebabkan oleh degradasi konteks pada *hidden state* akibat *vanishing gradient*—sekuens yang lebih panjang justru memaksa jaringan masuk ke dalam *repetitive loop* atau pengulangan kata yang redundan. Secara komputasional, hal ini mengakibatkan dilusi presisi *n-gram*; meskipun jumlah kata yang benar tetap, total jumlah token pada penyebut dalam perhitungan presisi terus bertambah, yang secara linear menurunkan nilai akhir metrik BLEU-4.

Dapat disimpulkan bahwa batasan $T_{\max} = 10$ merupakan titik *early stopping* paling optimal bagi distribusi probabilitas prediktif model pada dataset Flickr8k. Pada titik ini, model mampu mengompresi fitur visual ke dalam representasi leksikal yang padat tanpa terjebak dalam akumulasi galat sekuensial yang merusak presisi *n-gram* pada iterasi yang lebih panjang.

3. Kesimpulan dan Saran

3.1 Kesimpulan

Berdasarkan komparasi empiris dan analisis teoretis, arsitektur Long Short-Term Memory (LSTM) mendemonstrasikan keunggulan absolut dibandingkan SimpleRNN dalam memitigasi fenomena vanishing gradient selama proses Backpropagation Through Time (BPTT). Melalui interaksi matematis pada mekanisme gating dan aliran linear pada Cell State (C_t), LSTM terbukti mampu mempertahankan integritas gradien dan meretensi memori jangka panjang (long-term memory). Hal ini menghasilkan akurasi kualitatif translasi yang lebih tinggi serta mencegah mode collapse, meskipun arsitektur ini memicu lonjakan komputasi secara kuadratik akibat eskalasi operasi matriks. Terkait penskalaan hiperparameter (hyperparameter scaling), ekspansi kapasitas jaringan, baik melalui penambahan kedalaman arsitektur (>2 layer) maupun perluasan dimensi hidden state (dari 128 menjadi 512 unit), secara empiris menghasilkan diminishing returns. Konfigurasi berkapasitas masif ini memicu overhead komputasional yang parah pada saat iterasi greedy decoding dan memfasilitasi terjadinya overfitting prematur, tanpa memberikan kompensasi peningkatan yang proporsional pada metrik evaluasi BLEU-4 dan METEOR.

Terkait penskalaan hiperparameter (*hyperparameter scaling*), ekspansi kapasitas jaringan, baik melalui penambahan kedalaman arsitektur (>2 layer) maupun perluasan dimensi *hidden state* (dari 128 menjadi 512 unit), secara empiris menghasilkan *diminishing returns*. Konfigurasi berkapasitas masif ini memicu *overhead* komputasional yang parah pada saat iterasi *greedy decoding* dan memfasilitasi terjadinya *overfitting* prematur, tanpa memberikan kompensasi peningkatan yang proporsional pada metrik evaluasi BLEU-4 dan METEOR.

Eksperimen ini juga secara fundamental mengekspos batasan teoretis dari arsitektur dekoder *pre-inject*. Injeksi vektor konteks visual (x_{-1}) yang dieksekusi secara statis dan eksklusif pada *timestep* awal menyebabkan dekoder mengalami "amnesia temporal". Seiring dengan bertambahnya panjang sekuens waktu, model secara bertahap kehilangan *grounding* visual representasionalnya, memaksa dekoder untuk mengabaikan ruang fitur spasial dan mengandalkan probabilitas *n-gram* linguistik lokal semata.

3.2 Saran

Guna menyempurnakan performa translasi visual-leksikal pada eksperimen tingkat lanjut, diusulkan sejumlah optimasi arsitektural dan algoritmik sebagai berikut:

1. Peningkatan arsitektur untuk memitigasi kelemahan fundamental dari metode pre-inject melalui implementasi Attention Mechanism.

4. Pembagian Tugas Tiap Anggota Kelompok

NIM	Nama	Tugas yang Dikerjakan
13523156	Hasri Fayadh	Mengimplementasikan arsitektur Dense, LSTM Cell, dan Dense Projection from scratch. Melakukan pelatihan LSTM, analisis perbandingan Keras vs Scratch (LSTM), dan analisis komparatif performa/kualitas caption antara arsitektur RNN dan LSTM. Mengimplementasikan bonus Beam Search Decoder:
13523160	I Made Wiweka Putera	Membangun pipeline ekstraksi fitur gambar menggunakan pre-trained CNN (InceptionV3). Mengimplementasikan fungsionalitas preprocessing caption (Tokenisasi, Padding). Membangun arsitektur Embedding Layer dan Simple RNN from scratch secara modular dengan dukungan variasi num_layers dan hidden_dim. Melakukan eksperimen, pelatihan, perhitungan metrik (BLEU-4, METEOR), dan visualisasi loss untuk pipeline Image Captioning berbasis RNN.
18223121	Fudhail Fayyadh	Mengimplementasikan seluruh modul CNN from scratch (Conv2D, LocallyConnected2D, Pooling, Flatten, Activations). Mengimplementasikan utility functions (Image loader, Batch loader). Melakukan pelatihan, eksperimen (16 variasi), dan evaluasi Keras vs Scratch untuk task Image Classification.

5. Referensi

- https://d2l.ai/chapter_recurrent-modern/lstm.html
- https://d2l.ai/chapter_recurrent-modern/deep-rnn.html
- https://d2l.ai/chapter_recurrent-modern/bi-rnn.html
- https://d2l.ai/chapter_recurrent-neural-networks/index.html
- https://d2l.ai/chapter_convolutional-neural-networks/index.html
- <https://d2l.ai/index.html>
- https://d2l.ai/chapter_recurrent-modern/beam-search.html

6. Lampiran

Form Penggunaan AI

<https://drive.google.com/file/d/1i1r5bhI5sw0k3YnoGhVZLNiV74tEC-yf/view?usp=sharing>